

Modul 05: Directives & Pipes

<i>Ziel:</i>	<i>Templates formatieren</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~30min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>06.08.2024</i>

1 Todo Item aktualisieren und entfernen

Wenn das Todo Item abgehakt wird, soll es durchgestrichen werden. Die Priorität soll mit folgender Klasse formatiert werden:

```
.important::before {
  content: ' ! ';
  color: red;
}

.low::before {
  content: ' ↓ ';
  color: limegreen;
}
```

Wir können die Direktiven `ngStyle` und `ngClass` verwenden, um das Item zu formatieren.

```
<input type="checkbox" [checked]="item.completed"
      (change)="setCompleted(!item.completed)" />
<span
  class="title"
  [ngClass]="item.priority"
  [ngStyle]="{ 'text-decoration': item.completed ? 'line-through' : 'none' }"
  title="{{ item.id }}"
>
  {{ item.title }}
</span>
<button (click)="deleteTask(item.id)">🗑️</button>
```

Außerdem müssen wir dem Task-Service die Logik hinzufügen, das Item zu aktualisieren. In diesem Schritt können wir auch gleich die Logik hinzufügen es zu löschen.

```

updateTask(id: string, task: Partial<Omit<TaskItem, 'id'>>) {
  this.items.update((tasks) => {
    const index = tasks.findIndex((t) => t.id === id);
    if (index !== -1) {
      const updatedTask = { ...tasks[index], ...task };
      tasks.splice(index, 1, updatedTask);
    }
    return tasks;
  });
}

removeTask(id: string) {
  this.items.update((tasks) => tasks.filter((t) => t.id !== id));
}

```

2 Toasts abhängig vom Typ formatieren

Analog zu den Task Items sollen auch das Status Element abhängig vom Typ formatiert werden, also ob es sich um eine Informationen oder einen Fehler handelt. Folgende Klassen können verwendet werden:

```

.info {
  background-color: aliceblue;
  border: 1px solid gainsboro;
}

.error {
  background-color: lightpink;
  border: 1px solid darkred;
}

```

3 Fälligkeitsdatum hinzufügen

Die task-form Komponente soll um ein Input Fälligkeitsdatum ergänzt werden.

```
<label for="dueDate">Fälligkeit</label>
<input id="dueDate" #dueDateRef type="date" />
```

Das Datum kann mit der eingebauten Date-Pipe formatiert werden.

```
<span>(bis: {{ item.dueDate | date : 'yyyy-MM-dd' }})</span>
```

4 Relatives Fälligkeitsdatum anzeigen

Um uns die Arbeit mit dem Datum zu vereinfachen, installieren wir die Komponente luxon sowie ihre Typdefinition @types/luxon. Letzteres package sollte als Dev-Dependency installiert werden. Wir erstellen mit der CLI sodann eine eigene Pipe:

```
ng generate pipe pipes/relative-date
```

Die Implementierung sieht wie folgt aus:

```
import { DateTime } from 'luxon';

@Pipe({
  name: 'relativeDate',
})
export class RelativeDatePipe implements PipeTransform {
  transform(value: Date): string | null {
    return DateTime.fromJSDate(value).toRelative();
  }
}
```

So verwenden wir die eigene Pipe:

```
<span title="{{ item.dueDate | date : 'yyyy-MM-dd' }}">
  ({{ item.dueDate | relativeDate }})
</span>
```